调度算法

进程调度算法（CPU调度算法）

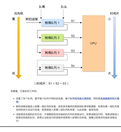
- 调度发生的条件
  - 进程从运行状态到等待状态
  - 进程从运行状态到就绪状态
  - 进程从等待状态到就绪状态
  - 进程从运行状态到终止状态
  - 1，4为非抢占式调度，2，3为抢占式调度
- 调度算法
  - 先来先服务调度算法
    - 每次从就绪队列里面选择最先进入队列的进程，然后一直运行，直到进程退出或被阻塞，才会继续从队列中选择第一个进程接着运行

    - 这种方法适合于CPU繁忙型作业的系统，而不适用于IO繁忙型作业的系统
  - 最短作业优先调度算法
    - 优先选择运行时间最短的进程来运行

    - 这种方法可能会导致长作业无法运行
  - 高响应比优先调度算法
    - 这个算法主要权衡了短作业和长作业，每次进行进程调度的时候，先计算响应比优先级，然后把响应比优先级最高的进程投入运行
    - 优先级计算公式：（等待时间＋要求服务时间）/要求服务时间
  - 时间片轮转调度算法
    - 每个进程被分配一个时间段，称为时间片，即允许该进程在时间段中运行
      - 如果时间片用完，进程还在运行，那么就会把此进程从cpu释放出来，并把cpu分配另外一个进程
      - 如果该进程在时间片结束之前阻塞或结束，则cpu进程切换
    - 时间片长度是一个很关键的点
      - 如果时间片舍得太短导致过多的进程上下文切换，降低了cpu效率

      - 如果设得太长又可能引起对短作业进程的响应时间变长，一般情况下时间片在20ms-50ms
  - 最高优先级调度算法
    - 从就绪队列中选择最高优先级的进程进行运行，这称为最高优先级调度算法
    - 优先级分类
      - 静态优先级：创建进程时候就确定的优先级，整个运行时间优先级都不会发生变化
      - 动态优先级：随着时间推移增加等待进程的优先级
    - 处理优先级的方法
      - 非抢占式：当就绪队列中出现优先级高的进程，运行完当前的进程，再选择优先级高的进程
      - 抢占式：当就绪队列中出现优先级高的进程，当前进程挂起，调度优先级高的进程运行
  - 多级反馈队列调度算法
    - 多级反馈队列是时间片轮转算法和最高优先级算法的综合和发展
      - 多级：表示多个队列，每个队列优先级从高到低，同时优先级越高时间片越短
      - 反馈：表示如果有新的进程加入优先级高的队列，立刻停止当前正在运行的进程，转而去运行优先级高的队列
    - 工作

子主题



内存页面置换算法

- 缺页中断的流程